VisorQA — Technical Design Specification

System Architecture, Database Schema, API Contracts & Meta AI Integration Blueprint

FILE: TECHNICAL_DESIGN.md
MODE: SOLO DEV (5-DAY SPRINT)
STACK: NEXT.JS + FASTAPI + PG

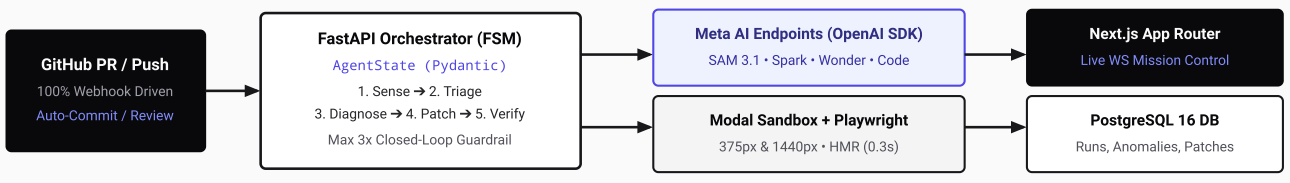
Next.js 15	Pure Python FSM	PostgreSQL 16	Meta AI APIs
App Router + WS Telemetry	FastAPI + Asyncio + Pydantic	SQLModel + AsyncPG Store	SAM 3.1 + Muse Suite

1. TECH STACK & PROJECT STRUCTURE (ZERO OVER-ENGINEERING)

- **Frontend (Mission Control UI):** `Next.js (App Router)`, TypeScript, Tailwind CSS, native browser `WebSocket` client, and Lucide icons. Built with a High-Craft Monochrome + AI Harmony aesthetic.
- **Backend Orchestrator:** `FastAPI` running a single-file Deterministic Finite State Machine (`orchestrator.py`) built strictly with `Python 3.11+`, `asyncio`, `Pydantic v2`, and `NumPy`. Zero agent frameworks (no LangChain/CrewAI) and zero Vector DBs.
- **Database & Storage:** `PostgreSQL` via `SQLModel` + `asyncpg` for non-blocking persistence of PR runs, math telemetry, and diffs; local static volume (`/static/masks/`) for PNG segmentation overlays.
- **AI & Sandbox Execution:** `Meta AI Endpoints` called via standard `AsyncOpenAI(base_url=...)` SDK with strict Pydantic `response_format` for **SAM 3.1**, **Muse Spark**, **Muse Wonder**, and **Muse Code**; `Modal Sandboxes` running headless `Playwright` and Next.js/React HMR.

```
visorqa/ ├── frontend/ # Next.js App Router (Mission Control Dashboard) │ └── src/app/ # page.tsx (Live Runs), runs/[id]/page.tsx (Inspector), components/ ├── backend/ # FastAPI + Pure Python Asyncio FSM │ ├── main.py # Webhook receiver (POST /api/webhooks/github), REST & WebSocket router │ ├── orchestrator.py # AgentState Pydantic model + 5 deterministic async nodes │ ├── vision_math.py # DOM-guided SAM 3.1 client + NumPy V_r, IoU, and Δ_x equations │ ├── github_client.py # HMAC-SHA256 verifier + PyGithub/HTTPX auto-commit & PR comment client │ └── db.py # PostgreSQL asyncpg engine & SQLModel table definitions └── modal_sandbox/ # Isolated Modal container runner with Playwright & HMR patcher
```

2. SYSTEM ARCHITECTURE & COMPONENT RESPONSIBILITIES



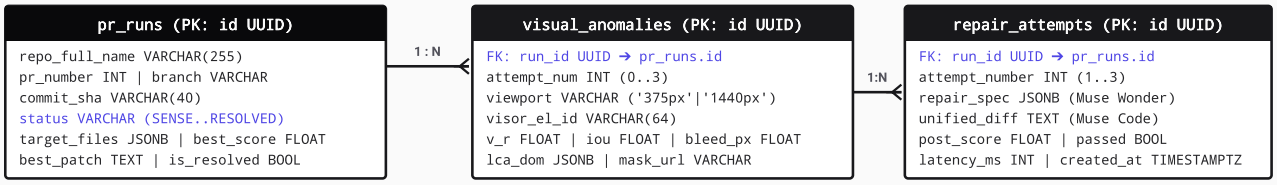
- **Node 1 —** `sense_with_playwright_and_sam(state)`**:** Renders the PR branch at `375px` and `1440px` in Modal, extracts interactive DOM boxes $\mathbf{B}_{\mathbf{DOM}}$ and center prompts $(\mathbf{c}_x, \mathbf{c}_y)$, calls SAM 3.1 via Meta Endpoint, and computes NumPy discrepancy metrics.
- **Node 2 —** `triage_files_muse_spark(state)`**:** Extracts the Lowest Common Ancestor (LCA) DOM subtree and computed CSS for failing elements, calling **Muse Spark** once to isolate the 1–2 parent layout files while protecting `src/components/ui/*`.
- **Node 3 —** `diagnose_muse_wonder(state)`**:** Sends the red SAM 3.1 collision mask overlay, math telemetry, and targeted file contents to **Muse Wonder** to produce a structured JSON `repair spec`.

3. DETERMINISTIC VISION MATH & LCA TRIAGE ENGINE

> **Governing Pixel Discrepancy Equations (NumPy Matrix Evaluation in vision_math.py)**
> Every interactive element receives a unique `visor-el-id`. Its center (c_x, c_y) and DOM bounding box B_{DOM} prompt SAM 3.1 to yield binary pixel mask M_{SAM}. An anomaly is flagged deterministically if any of the following three thresholds trigger:
>
> - **1. Z-Index Occlusion (Visibility Ratio):** $V_r = Area(M_{SAM}) \div Area(B_{DOM})$ → **Defect Trigger:** $V_r < 0.85$ (Verification Target: $V_r \geq 0.98$)
> - **2. Sibling Collision (Intersection over Union):** $IoU(i, j) = Area(M_i \cap M_j) \div Area(M_i \cup M_j)$ → **Defect Trigger:** $IoU > 0.05$ (Verification Target: $IoU = 0.00$)
> - **3. Viewport Bleed (Horizontal Overflow):** $\Delta_x = max(0, X_{max}(B_{DOM}) - Width_{Viewport})$ → **Defect Trigger:** $\Delta_x > 2px$ (Verification Target: $\Delta_x \leq 2px$)

- **Composite Health Score Calculation:** After each verification pass across Mobile (`375px`) and Desktop (`1440px`), the orchestrator computes $Score = mean(V_r) - max(IoU) - (0.01 \times \Delta_x)$. Whenever `score > state.best_score` , the FSM snapshots `state.best_patch = state.current_patch` .
- **LCA DOM Extraction Algorithm:** Playwright traverses upward from the colliding/clipped DOM nodes using `element.parentElement` until finding the first shared parent container, serializing its tag hierarchy, class names, and computed layout styles (`display` , `position` , `z-index` , `flex-wrap` , `overflow`) into a compact 3.2K-token payload for Muse Spark.

4. POSTGRESQL DATABASE SCHEMA & ER DIAGRAM



- **Table 1 — `pr_runs` :** Stores the master state of every GitHub Webhook execution, mirroring `AgentState` in PostgreSQL so the Next.js App Router can hydrate historical and active runs via SSR or REST, indexed on `(repo_full_name, pr_number)` .
- **Table 2 — `visual_anomalies` :** Records each failing interactive element per viewport and attempt, storing exact NumPy metrics (`v_r` , `iou` , `bleed_px`), the serialized LCA DOM subtree (`JSONB`), and paths to the before/after SAM 3.1 mask PNGs.
- **Table 3 — `repair_attempts` :** Provides full auditability of the 3-Strike Self-Healing Loop, storing Muse Wonder's structured `repair_spec` JSONB, Muse Code's `unified_diff` , verification score delta, and HMR execution latency in milliseconds.

5. FRONTEND–BACKEND API & WEBSOCKET TELEMETRY CONTRACTS

The Next.js App Router communicates with the FastAPI backend through typed REST endpoints for initial page hydration and a single multiplexed WebSocket connection for sub-second FSM state transitions.

Protocol	Endpoint / Channel	Request / Trigger	Response / Broadcast Contract
`POST`	`/api/webhooks/github`	GitHub `pull_request` or `push` event + `X-Hub-Signature-256`	`202 Accepted` `{"run_id": "uuid", "status": "QUEUED"}`; spawns async `run_visor_agent()` task.
`GET`	`/api/runs`	Query: `?limit=20&status=all`	`200 OK` Array of `PRRunSummary` objects from PostgreSQL for Mission Control sidebar.
`GET`	`/api/runs/{run_id}`	Path UUID `run_id`	`200 OK` Full `AgentState` snapshot including `visual_anomalies` and `repair_attempts`.
`WS`	`/ws/runs/{run_id}`	Event: `SENSE_COMPLETE`	Streams initial `375px` & `1440px` screenshots, SAM 3.1 mask URLs, and failing $\mathbf{V_r}$, $\mathbf{IoU}$, $\mathbf{\Delta_x}$ array.
`WS`	`/ws/runs/{run_id}`	Event: `TRIAGE_COMPLETE`	Streams LCA DOM hierarchy and `target_files: list[str]` isolated by Muse Spark in 0.6s.
`WS`	`/ws/runs/{run_id}`	Event: `ATTEMPT_{1..3}_FEEDBACK`	Streams Muse Wonder `repair_spec`, Muse Code `current_patch` diff, HMR reload time, and updated SAM scores.

```
      // WebSocket Broadcast Envelope (Sent via await ws.send_json(...) at every FSM Node transition) { "event": "ATTEMPT_1_FEEDBACK", "timestamp": "2026-10-27T08:00:11.420Z", "state": { "run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d", "pr_url": "https://github.com/org/storefront/pull/42", "attempt": 1, "max_attempts": 3, "is_resolved": true, "best_score": 0.994, "target_files": ["src/app/checkout/CheckoutPage.tsx"], "repair_spec": {"defect_type": "Z_INDEX_OCCLUSION", "root_cause": "Sticky banner overlaps CTA", "fix_strategy": "Add pb-24 to parent flex container and scope banner z-index"}, "current_patch": "--- a/src/app/checkout/CheckoutPage.tsx\n+++ b/src/app/checkout/CheckoutPage.tsx..." } }
```

6. AUTHENTICATION, GITHUB WEBHOOKS & EXTERNAL INTEGRATIONS

- **100% Autonomous GitHub Webhook Security (No Manual UI Triggers):** In strict compliance with ADR-001, Mission Control contains zero manual trigger buttons. Execution begins exclusively when GitHub dispatches a signed webhook to `POST /api/webhooks/github`. FastAPI validates the payload using `hmac.compare_digest` with `GITHUB_WEBHOOK_SECRET` (`X-Hub-Signature-256`).
- **GitHub Commit & Fallback Review API:** Authenticated via fine-grained `GITHUB_PAT` (`contents:write`, `pull_requests:write`). When `state.is_resolved == True`, `github_client.py` commits `state.best_patch` directly to the PR branch via the Git Trees/Commits API and posts a Markdown before/after visual diff table. If 3 attempts fail to reach $\mathbf{V_r} \geq \mathbf{0.98}$ and $\mathbf{IoU} = \mathbf{0}$, it posts `state.best_patch` as a non-destructive GitHub Pull Request Review suggestion.
- **Meta AI Endpoints via Standard OpenAI SDK:** All three Muse models (**Muse Spark** for LCA file triage, **Muse Wonder** for multimodal layout diagnosis, and **Muse Code** for patch synthesis) and **SAM 3.1** are invoked via `AsyncOpenAI(api_key=META_API_KEY, base_url=META_BASE_URL)` using